

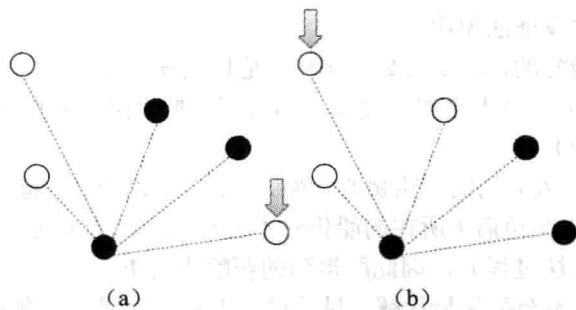


图 8-6 巨人与鬼问题

8.4 贪心法

贪心法是一种解决问题的策略。如果策略正确，那么贪心法往往是易于描述、易于实现的。本节介绍可以用贪心法解决的若干经典问题。

8.4.1 背包相关问题

最优装载问题。给出 n 个物体，第 i 个物体重量为 w_i 。选择尽量多的物体，使得总重量不超过 C 。

【分析】

由于只关心物体的数量，所以装重的没有装轻的划算。只需把所有物体按重量从小到大排序，依次选择每个物体，直到装不下为止。这是一种典型的贪心算法，它只顾眼前，但却能得到最优解。

部分背包问题。有 n 个物体，第 i 个物体的重量为 w_i ，价值为 v_i 。在总重量不超过 C 的情况下让总价值尽量高。每一个物体都可以只取走一部分，价值和重量按比例计算。

【分析】

本题在上一题的基础上增加了价值，所以不能简单地像上题那样先拿轻的（轻的可能价值也小），也不能先拿价值大的（可能它特别重），而应该综合考虑两个因素。一种直观的贪心策略是：优先拿“价值除以重量的值”最大的，直到重量和正好为 C 。

注意：由于每个物体可以只拿一部分，因此一定可以让总重量恰好为 C （或者全部拿走重量也不足 C ），而且除了最后一个以外，所有的物体要么不拿，要么拿走全部。

乘船问题。有 n 个人，第 i 个人重量为 w_i 。每艘船的最大载重量均为 C ，且最多只能乘两个人。用最少的船装载所有人。

【分析】

考虑最轻的人 i ，他应该和谁一起坐呢？如果每个人都无法和他一起坐船，则唯一的方案就是每人坐一艘船（想一想，为什么）。否则，他应该选择能和他一起坐船的人中最重的一个 j 。这样的方法是贪心的，因此它只是让“眼前”的浪费最少。幸运的是，这个贪心

策略也是对的，可以用反证法说明。

假设这样做不是最好的，那么最好方案中 i 是什么样的呢？

情况 1: i 不和任何一个人坐同一艘船，那么可以把 j 拉过来和他一起坐，总船数不会增加（而且可能会减少）。

情况 2: i 和另外一人 k 同船。由贪心策略， j 是“可以和 i 一起坐船的人”中最重的，因此 k 比 j 轻。把 j 和 k 交换后 k 所在的船仍然不会超重（因为 k 比 j 轻），而 i 和 j 所在的船也不会超重（由贪心法过程），因此所得到的新解不会更差。

由此可见，贪心法不会丢失最优解。最后说一下程序实现。在刚才的分析中，比 j 更重的人只能每人坐一艘船。这样，只需用两个下标 i 和 j 分别表示当前考虑的最轻的人和最重的人，每次先将 j 往左移动，直到 i 和 j 可以共坐一艘船，然后将 i 加 1， j 减 1，并重复上述操作。不难看出，程序的时间复杂度仅为 $O(n)$ ，是最优算法（别忘了，读入数据也需要 $O(n)$ 时间，因此无法比这个更好了）。

8.4.2 区间相关问题

选择不相交区间。数轴上有 n 个开区间 (a_i, b_i) 。选择尽量多个区间，使得这些区间两两没有公共点。

【分析】

首先明确一个问题：假设有两个区间 x, y ，区间 x 完全包含 y 。那么，选 x 是不划算的，因为 x 和 y 最多只能选一个，选 x 还不如选 y ，这样不仅区间数目不会减少，而且给其他区间留出了更多的位置。接下来，按照 b_i 从小到大的顺序给区间排序。贪心策略是：一定要选第一个区间。为什么？

现在区间已经排序成 $b_1 \leq b_2 \leq b_3 \dots$ 了，考虑 a_1 和 a_2 的大小关系。

情况 1: $a_1 > a_2$ ，如图 8-7 (a) 所示，区间 2 包含区间 1。前面已经讨论过，这种情况下一定不会选择区间 2。不仅区间 2 如此，以后所有区间中只要有一个 i 满足 $a_1 > a_i$ ， i 都不要选。在今后的讨论中，将不考虑这些区间。

情况 2: 排除了情况 1，一定有 $a_1 \leq a_2 \leq a_3 \leq \dots$ ，如图 8-7 (b) 所示。如果区间 2 和区间 1 完全不相交，那么没有影响（因此一定要选区间 1），否则区间 1 和区间 2 最多只能选一个。如果不选区间 2，黑色部分其实是没有任何影响的（它不会挡住任何一个区间），区间 1 的有效部分其实变成了灰色部分，它被区间 2 所包含！由刚才的结论，区间 2 是不能选的。依此类推，不能因为选任何区间而放弃区间 1，因此选择区间 1 是明智的。

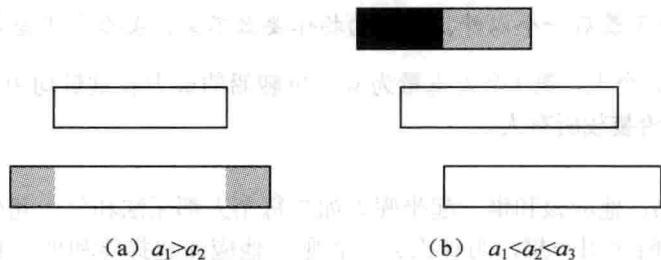


图 8-7 贪心策略图示

选择了区间 1 以后, 需要把所有和区间 1 相交的区间排除在外, 需要记录上一个被选择的区间编号。这样, 在排序后只需要扫描一次即可完成贪心过程, 得到正确结果。

区间选点问题。数轴上有 n 个闭区间 $[a_i, b_i]$ 。取尽量少的点, 使得每个区间内都至少有一个点 (不同区间内含的点可以是同一个)。

【分析】

如果区间 i 内已经有一个点被取到, 则称此区间已经被满足。受上一题的启发, 下面先讨论区间包含的情况。由于小区间被满足时大区间一定也被满足, 所以在区间包含的情况下, 大区间不需要考虑。

把所有区间按 b 从小到大排序 (b 相同时 a 从大到小排序), 则如果出现区间包含的情况, 小区间一定排在前面。第一个区间应该取哪一个点呢? 此处的贪心策略是: 取最后一个点, 如图 8-8 所示。

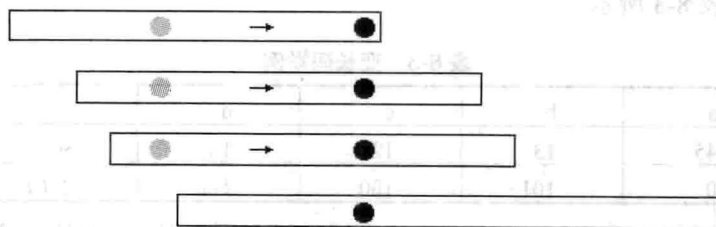


图 8-8 贪心策略

根据刚才的讨论, 所有需要考虑的区间的 a 也是递增的, 可以把它画成图 8-8 的形式。如果第一个区间不取最后一个, 而是取中间的, 如灰色点, 那么把它移动到最后一个点后, 被满足的区间增加了, 而且原先被满足的区间现在一定被满足。不难看出, 这样的贪心策略是正确的。

区间覆盖问题。数轴上有 n 个闭区间 $[a_i, b_i]$, 选择尽量少的区间覆盖一条指定线段 $[s, t]$ 。

【分析】

本题的突破口仍然是区间包含和排序扫描, 不过先要进行一次预处理。每个区间在 $[s, t]$ 外的部分都应该预先被切掉, 因为它们的存在是毫无意义的。预处理后, 在相互包含的情况下, 小区间显然不应该考虑。

把各区间按照 a 从小到大排序。如果区间 1 的起点不是 s , 无解 (因为其他区间的起点更大, 不可能覆盖到 s 点), 否则选择起点在 s 的最长区间。选择此区间 $[a_i, b_i]$ 后, 新的起点应该设置为 b_i , 并且忽略所有区间在 b_i 之前的部分, 就像预处理一样。虽然贪心策略比上题复杂, 但是仍然只需要一次扫描, 如图 8-9 所示。 s 为当前有效起点 (此前部分已被覆盖), 则应该选择区间 2。

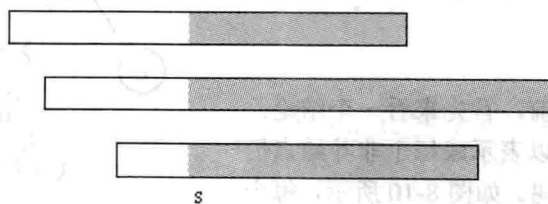


图 8-9 区间覆盖问题

8.4.3 Huffman 编码

假设某文件中只有 6 种字符：a, b, c, d, e, f，可以用 3 个二进制位来表示，如表 8-2 所示（表 8-2~表 8-4 中，频率的单位均为“千次”）。

表 8-2 各种字符的编码

字 符	a	b	c	d	e	f
频 率	45	13	12	16	9	5
编 码	000	001	010	011	100	101

这样，一共需要 $(45+13+12+16+9+5)*3=300$ 千比特（即二进制的位）。第二种方法是采用变长编码，如表 8-3 所示。

表 8-3 变长码举例

字 符	a	b	c	d	e	f
频 率	45	13	12	16	9	5
编 码	0	101	100	111	1101	1100

总长度为 $1*45+3*13+3*12+3*16+4*9+4*5=224$ 千比特，比定长码短。读者可能会说：还可以更短，如表 8-4 所示。

表 8-4 错误的变长码举例

字 符	a	b	c	d	e	f
频 率	45	13	12	16	9	5
编 码	0	1	00	01	10	11

总长度只有 $1*(45+13)+2*(12+16+9+5)=142$ 千比特，不是更短吗？可惜，这样的编码方案是有问题的。如果收到了 001，那么究竟是 aab、cb，还是 ad？换句话说，这样的编码有歧义，因为其中一个字符的编码是另一个码的前缀（prefix）。表 8-3 所示的码没有这样的情况，任何一个编码都不是另一个的前缀。这里把满足这样性质的编码称为前缀码（Prefix Code）。下面正式叙述编码问题。

最优编码问题。给出 n 个字符的频率 c_i ，给每个字符赋予一个 01 编码串，使得任意一个字符的编码不是另一个字符编码的前缀，而且编码后总长度（每个字符的频率与编码长度乘积的总和）尽量小。

【分析】

在解决这个问题之前，首先来看一个结论：任何一个前缀编码都可以表示成每个非叶结点恰好有两个子结点的二叉树。如图 8-10 所示，每个非叶结点与左子结点的边上写 1，与右子结点的

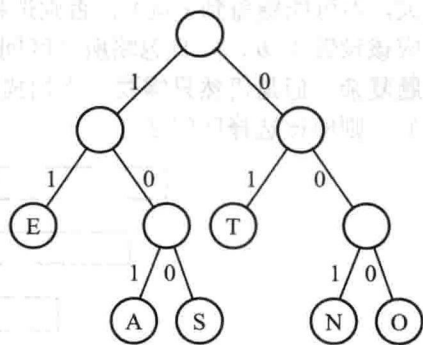


图 8-10 前缀码的二叉树表示